Virtualization-cpu-context

cpu的虚拟化（中）——— 上下文

- 前言
- 上下文初识
- 上下文的具体内容
- 上下文切换

前言

- 在前一篇笔记中，主要介绍了进程的相关问题，接下来要聊的是**上下文**，正文还是以围绕“是什么”与“为什么”进行展开，笔记中若有错漏，望不吝指正。

上下文初识

上下文这是一个与进程密切相关的对象。上下文与进程的关系可以这样描述：**每个进程都有自己的上下文**。什么意思呢？主要强调了两点：

1. 没有一个进程是没有上下文的。就 ：正常情况下，没有一个人是没有手脚的。上下文是进程天然拥有的属性。
2. 不存在A进程拥有B进程上下文的情况。还是用上一个例子：正常情况下，没有两个人是共用一副手脚的。上下文是进程所私有的。

上下文的具体内容

以上只是对上下文定了性，为了更全面的了解上下文，我们聊聊更具体的情况，即上下文包含哪些内容[1]？在CPU中，进程的一部分上下文存储在一个个寄存器中，寄存器就是放在CPU内部，用来存储数据的小盒子。

1. **PC寄存器**：
   - 该寄存器用于存储进程下一条要执行指令的地址。
   - 示例：PC寄存器的值是0x7fffe000，则CPU执行的下一条指令的地址是0x7fffe000。
2. **通用寄存器组**：
   - 该组中包含多个寄存器，这些寄存器用来存储程序在运行的过程（即进程）产生的临时数据或中间计算结果。
3. **栈指针寄存器**：
   - 栈指针寄存器中存储的是当前进程的用户栈中栈顶元素的地址。

4. **标志寄存器**
    - 记录了执行算术运算或位运算后，产生的标志。
    - 用途：主要用于完成关系运算， ：>,<,<=,>=等。

综上，一个进程的一部分上下文中的具体内容便储存在这些CPU内部的寄存器中，这些名称也反映了其功能。

上下文切换

在上下文有了一个基本的认识之后。现在便轮到了上下文切换了。此时，我们可能会想上下文又没招惹谁，为什么要切换它，这便引出了第一个问题————为什么要进行上下文切换。接下来，假定我们知道了进行上下文切换的原因，那么，一个新的疑惑便浮现了出来，它便是上下文切换是 何实现的？为了解释清楚这些事情，我不得不要先说内核模式，因为，现代计算机的上下文都是在内核模式下完成的。

1. CPU的运行模式：
    - **用户模式**：
        i. 介绍：用户模式是一种受限的模式。在这个模式下有一部分机器指令是不允许被执行的，这些指令有个名字————特权指令。一旦在此 模式的某个进程执行了特权指令，就会触发异常，转由操作系统接手对该违规进程的处理。
        ii. 示例：用汇编语言描述的传送指令（mov,lea…）、算术指令（add,sub…）等等。
    - **内核模式**：
        i. 介绍：内核模式是一种不受限的模式。在这种模式下可以执行所有的机器指令。
        ii. 示例：用汇编语言描述的中断控制指令（cli,sti…)、等等。
2. 为什么需要两种不同的指令执行模式?
    - 安全性：这种设计有利于计算机安全的运行。可以避免恶意的进程直接执行有些指令对计算机造成不可修复的破坏。例 ：一个恶意的进程 直接窃取用户计算机中的所有信息，挟持 屏幕、键盘、音响等硬件的控制权。
    - 类比：开发者编写的程序就像是一个小孩，计算机是它的家，操作系统是家里的大人。在生活里大人是不会允许小孩接触 ：打火机、刀具 、电线等危险物品的。这种接触既可能伤害自己，也有可能会伤害他人。因此，便需要一个东西将这些危险物品隔离起来。
3. 模式切换的触发条件
    - **硬件中断**
        i. 每隔一段时间硬件会产生一个中断信号，CPU接收到信号之后会暂停当

前进程，然后由CPU保存进程的PC寄存器、标志寄存器等会被后 续指令修改内容的寄存器到进程的内核栈中，再执行模式切换指令进入内核模式。内核栈是一片处于操作系统管理的内核空间中，分配给 每个进程的内核模式私有栈，用于执行由进程触发的各种系统调用。在这里读者可能会有一个疑问，那就是为什么不使用进程本身的用户 栈来完成系统调用呢？因为，用户栈对用户进程而言是可写可读的，所以，系统调用可能受到恶意程序的攻击，造成严重的后果。最后， 至于为什么会设计周期性的中断信号？这个问题将在下文中解释。

- **系统调用**
  i. 系统调用是操作系统提供给用户进程的一系列函数。它的特殊之处在于所有的系统调用都在内核模式下执行，它的产生是为了给用户 进程提供它们自己不能做但又需要做的行为，沿用之前的一个例子：当小孩（用户进程）需要用火烤辣条时，由于火焰于它而言太危险， 因此，它不能用火。那此时该怎么办呢？这时大人（操作系统）就粉墨登场了。它用火烤好了辣条，然后递给了小孩。这样既满足了小孩 的心愿，又避免了小孩直接使用火焰。
- **异常**
  i. 在程序运行的过程中，程序有可能会出现一些非正常的行为，比 ：对0执行除法、用户栈溢出、访问违规的地址等等。当程序做出这 种行为时，CPU就会暂停当前进程，然后进入内核模式，执行预设的异常处理程序。

以上简要介绍了一下内核模式，然后让我们开始聊聊本节的重头戏————上下文切换。

4. 上下文切换的触发条件
   - **主动让出**
     i. 使用sleep()等系统调用使进入阻塞状态。
     ii. 使用sched_yield()或pthread_yield()等函数，通知调度器让出CPU，进程进入就绪状态。
   - **进程阻塞**
     i. 进程在等待网络请求或外部数据等不可立即获取的资源时会进入阻塞状态，此时调度器会选择其他的就绪进程使用CPU，从而避免CPU空闲。
   - **优先级抢占**
     i. 调度器在调度进程的依据就是为其设置的优先级，若A进程在运行状态时，突然加入了一个更高优先级的进程。此时，调度器将让更高优先级的进程使用CPU。
   - **时间片耗尽**
     i. 调度程序为每个进程赋予了一个时间值，用来规定每个进程可以使用CPU的时间，这便是时间片。为了完成对进程所剩时间片的检查。

从上文的触发条件可以看出进程可能因执行某些指令从而主动让出CPU，也有可能因新加入了更高优先级的进程从而被抢占。前者我们或许还可以理解，但后者却有点摸不着头脑，一个进程执行的好好的，调度器是怎么知道新来了一个更高优先级的进程，从而完成调度的？这其实就是设计周期性中断的原因了，有了周期性的中断之后，不管是谁在使用CPU，它会不会主动让出CPU的使用权，操作系统都可以定期的获取CPU的使用权，来重新决定运行哪个进程。

5. 上下文的切换过程
   - **切换环境**
     i. 任何的上下文切换都是在内核模式下完成的。从上文中的触发条件可以看出这些条件都会触发模式切换。
     ii. 原因：所有的上下文触发条件都会先触发模式切换。
   - **保存位置**
     i. 旧进程的上下文会被保存到对应的PCB中。
   - **切换实现**
     i. 保存旧进程的寄存器到PCB中，其中包括：栈指针寄存器、pc寄存器、通用寄存器、标志寄存器等。
     ii. 切换页表基址寄存器的值为新进程的页表值，从而完成虚拟空间的切换。
     iii. 从新进程的PCB中加载新进程的寄存器值，切换内核栈为新进程的内核栈。
     iv. 将旧进程的状态切换为阻塞/就绪状态（根据触发的条件不同），更新新进程的状态为运行。

---

1. 上文涉及寄存器的部分，因为，CPU架构的不同（x86-64，ARM等）带来的命名与使用约定的不同。所以，只谈了寄存器存储数据的作用，不涉及具体的寄存器名称与使用规定。 ↩